

Systèmes Informatiques

9. Systèmes d'exploitation – les systèmes de fichiers

Guillaume Pierre

ISTIC, L3 MIAGE





IBM en train de livrer un disque dur de 5 MB (1956)...

Table of Contents

1 Introduction

2 Implémentation des systèmes de fichiers

3 Technologies RAID

4 Conclusion

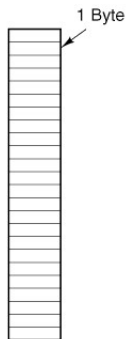
- Vos programmes utilisent le **stockage persistant**
 - On veut garder les données même après avoir éteint l'ordinateur
 - Potentiellement de très grands volumes de données
- Solution: **les fichiers** (évidemment)
- Question: **comment un OS gère-t-il les fichiers?**
 - Structure
 - Nommage
 - Contrôle d'accès
 - Protection
 - Implémentation
 - etc.

- Combien de caractères?
 - Par exemple: 8+3 ou 255 caractères
- Sensibles à la casse?
 - `maria` vs. `Maria` vs. `MARIA`
- Caractères spéciaux?
 - Exemple: `2?,slidês.pdf`
- Extensions de noms pour indiquer le type de fichier
 - Exemple: `slides.pdf`, `slides.pdf.gz`
 - Est-ce que l'OS doit gérer directement ces extensions de noms de fichier?

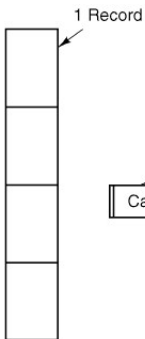
- Tous les OS supportent les **fichiers normaux** et les **répertoires**
- Il existe d'autres types de fichiers:
 - **Fichiers spéciaux "caractères"**
 - Une abstraction commode pour représenter des périphériques adressables par caractères (souris, clavier...)
 - **Fichiers spéciaux "blocs"**
 - Une abstraction commode pour représenter des périphériques adressables par blocs (disques...)
 - Fichiers de métadonnées (Windows)
 - Etc.
- **Fortement typés** (MacOS) ou **faiblement typés** (Linux)

Organisation d'un fichier

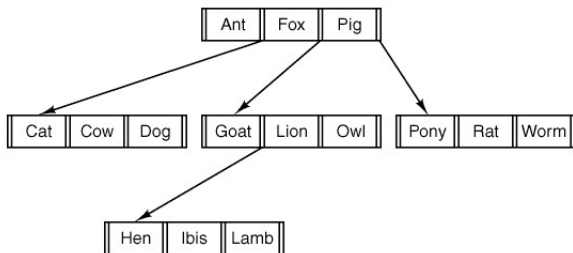
- Une séquence d'**octets**?
- Une séquence d'**enregistrements**?
- **Autre chose**?



(a)



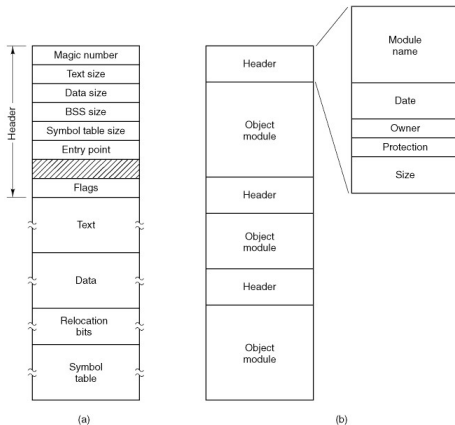
(b)



(c)

Fichiers réguliers == séquence d'octets

- Dans les OS modernes: **fichier régulier == séquence d'octets**
- Mais ça n'empêche pas de donner une **structure interne** à la séquence d'octets...



Fichier exécutable

Fichier archive

Attributs d'un fichier

- On attache des **attributs** à chaque fichier qui contiennent des méta-données à propos du fichier

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

1 Créer un nouveau fichier vide

2 Supprimer un fichier

3 Ouvrir un fichier

- L'OS lit les attributs du fichier et détermine sa localisation sur disque avant de donner (ou non) accès au fichier

4 Fermer

- L'OS libère les informations qu'il gardait en mémoire à propos du fichier (attributs, position dans le fichier, ...)

5 Lire

- Généralement: lire *depuis la position actuelle*

6 Écrire

- Depuis la position actuelle (en écrasant éventuellement des contenus)

7 Rajouter

- Ecrire à la fin du fichier

8 Déplacer

- Changer de position dans le fichier

9 Lire les attributs

10 Changer les attributs

11 Renommer le fichier

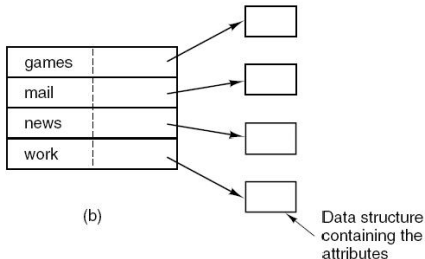
12 Verrouiller/déverrouiller

- Pour s'assurer qu'aucun autre processus n'accède au fichier

- Un répertoire est un fichier (presque) comme les autres
 - Il contient une liste de fichiers...
 - ... et leurs attributs

games	attributs
mail	attributs
news	attributs
work	attributs

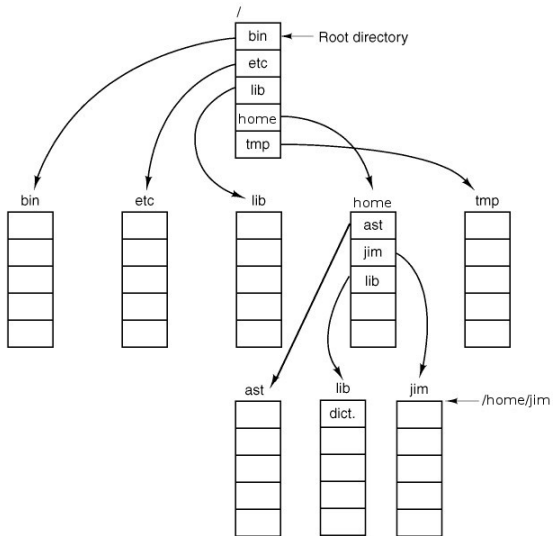
(a)



(b)

- Un répertoire peut contenir les noms/attributs d'autres répertoires
 - Structure hiérarchique
 - Chemins: `\home\gpierre\mailbox` ou `/home/gpierre/mailbox` ou `../../mailbox`

Arborescence standard des systèmes Unix



1 Créer un répertoire vide

- Les entrées spéciales `.` et `..` sont créées automatiquement

2 Supprimer

- Possible seulement si le répertoire est vide! (on ne veut pas perdre de fichiers)

3 Opendir

4 Closedir

5 Readdir

- Lit **une entrée** du répertoire

6 Renommer

- Renomme le répertoire

7 Link/unlink

- Permet à **un même fichier** (pas des copies du fichier!) d'apparaître dans plusieurs répertoires à la fois

Table of Contents

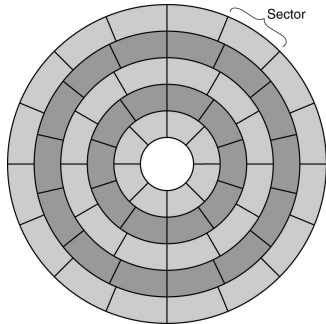
1 Introduction

2 Implémentation des systèmes de fichiers

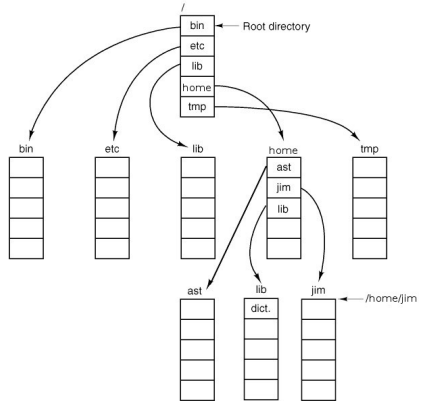
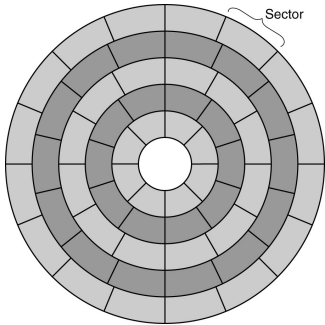
3 Technologies RAID

4 Conclusion

Implémentation des systèmes de fichiers

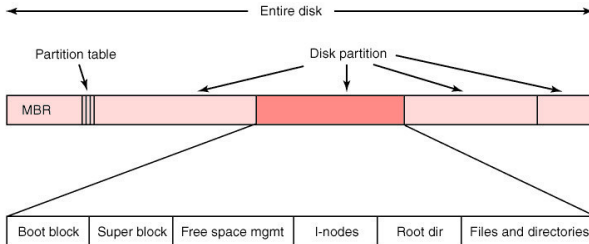


Implémentation des systèmes de fichiers



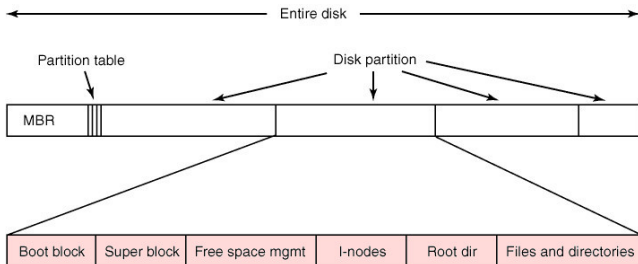
Organisation d'un disque

- Les disques sont généralement **partitionnés**
- **MBR = Master Boot Record**
 - Utile pour démarrer l'ordinateur
 - Contient (or pointe vers) la **table de partition**:
 - Numéro de bloc de début/fin de la partition
 - Type de système de fichiers
- Un système de fichier utilise **une seule partition**



Organisation d'un système de fichiers

- Tous les systèmes de fichiers commencent par un **boot block**
 - Vide si le système de fichier n'est pas bootable
- **Tout le reste dépend du système de fichiers**
 - Par exemple, dans Unix le **superblock** contient les paramètres de configuration du système de fichier
 - Un même OS peut supporter plusieurs types de systèmes de fichiers: bfs (BeOS), cramfs (Compressed RAM), ext2, ext3, ext4, minix, msdos, reiserfs, vfat (Windows FAT16, FAT32), iso9660 (CD-ROM) etc.

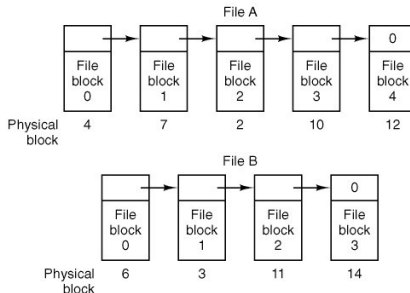


- Nous devons garder une trace des **correspondances fichier/blocs**
 - Quels sont les blocs qui constituent un fichier? Dans quel ordre?
- Une idée très simple: implémentation contigüe
 - Chaque fichier commence à partir d'un certain numéro de bloc
 - Chaque fichier utilise ***n* blocs contigus**
 - **Bien**: accès très efficace au fichier
 - **Bien**: il est facile de se déplacer dans le fichier
 - **Très très mauvais**:
 - 1 Les fichiers ne peuvent pas grandir
 - 2 Si les fichiers se rétrécissent (ou sont supprimés) cela crée des trous entre les fichiers
 - Question: dans quelles conditions cette stratégie est-elle utile?

- Nous devons garder une trace des **correspondances fichier/blocs**
 - Quels sont les blocs qui constituent un fichier? Dans quel ordre?
- Une idée très simple: implémentation contigüe
 - Chaque fichier commence à partir d'un certain numéro de bloc
 - Chaque fichier utilise **n blocs contigus**
 - **Bien**: accès très efficace au fichier
 - **Bien**: il est facile de se déplacer dans le fichier
 - **Très très mauvais**:
 - 1 Les fichiers ne peuvent pas grandir
 - 2 Si les fichiers se rétrécissent (ou sont supprimés) cela crée des trous entre les fichiers
 - Question: dans quelles conditions cette stratégie est-elle utile?
Systèmes de fichier en lecture seule
(par ex: CD-ROM)

Allocation de blocs par liste chaînée

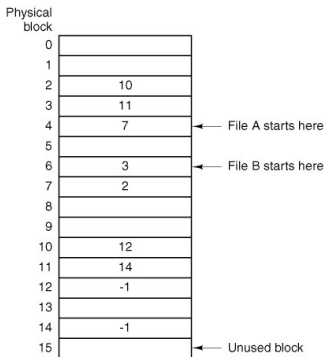
- Idée: faire démarrer chaque bloc de données par l'identifiant du **prochain bloc** dans le fichier
 - Le dernier bloc du fichier contient un pointeur nul
 - Le reste du bloc contient les données du fichier



- **Bien:** les fichiers peuvent être redimensionnés à la demande
- **Bien:** l'accès séquentiel au fichier est assez efficace
- **Mauvais:** déplacements coûteux dans le fichier
- **Bizarre:** les données sont stockées par unités bizarre (ex: 508 octets)
 - Les programmes lisent/écrivent souvent les données par puissance de 2 (par exemple: 1024 octets)

Améliorons la structure par liste chaînée

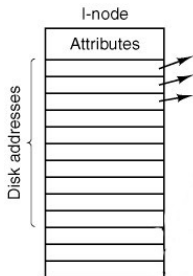
- Rajoutons des méta-données: une **File Allocation Table (FAT)**
 - Une seule FAT pour l'ensemble du système de fichiers
 - La FAT sépare la liste chaînée (méta-données) des données du fichier



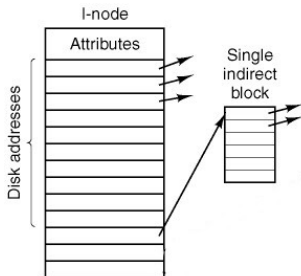
- **Bien:** Les déplacements dans le fichier sont plus rapides
- **Pas très bien:** il faut stocker la FAT en mémoire
500 GB = 500 millions de blocs de 1-KB

Les systèmes de fichiers Windows marchent comme ça (FAT16, FAT32)...

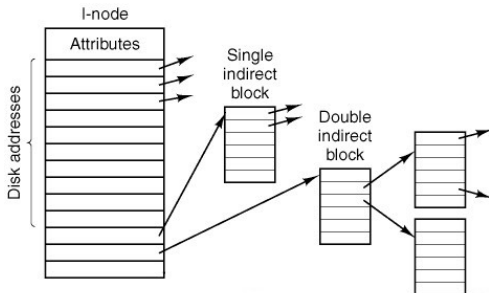
- Un **inode** (index-node) est un bloc qui contient les méta-informations d'un fichier, y compris la liste de ses blocs de données
 - Un seul inode par fichier
 - Un inode peut stocker n adresses de blocs de données



- Un **inode** (index-node) est un bloc qui contient les méta-informations d'un fichier, y compris la liste de ses blocs de données
 - Un seul inode par fichier
 - Un inode peut stocker n adresses de blocs de données

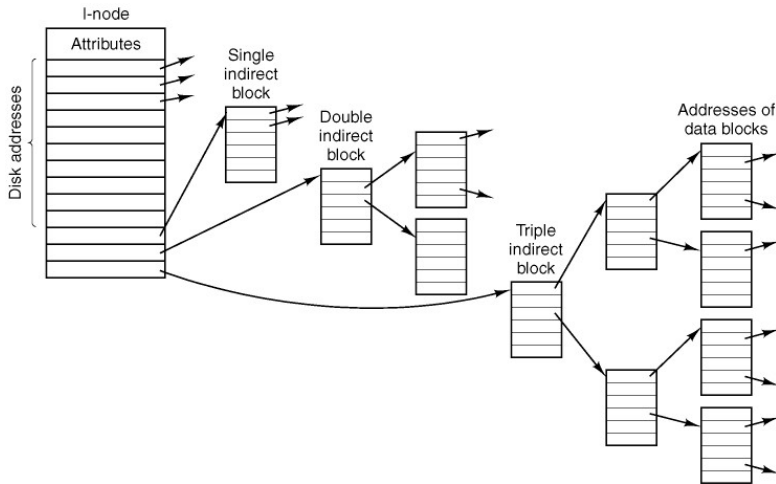


- Un **inode** (index-node) est un bloc qui contient les méta-informations d'un fichier, y compris la liste de ses blocs de données
 - Un seul inode par fichier
 - Un inode peut stocker n adresses de blocs de données



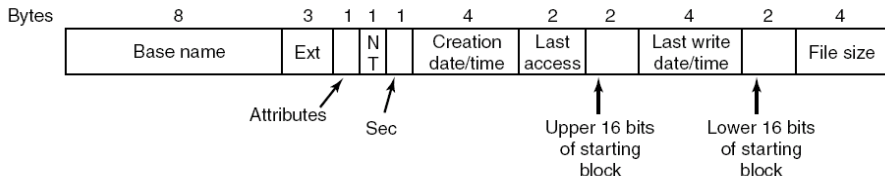
Inodes

- Un **inode** (index-node) est un bloc qui contient les méta-informations d'un fichier, y compris la liste de ses blocs de données
 - Un seul inode par fichier
 - Un inode peut stocker n adresses de blocs de données



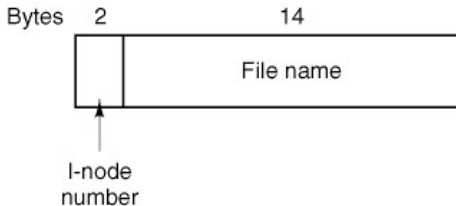
Implémentation des répertoires dans Windows

- Les vieux systèmes de fichiers Windows supportent des noms de fichiers sur **8 caractères + 3 caractères d'extension** (par ex: foobarba.pdf)



(Il y a une astuce pour permettre des noms de fichiers plus longs...)

- Les entrées du répertoire sont très simples:



- Toutes les méta-informations au sujet du fichier sont stockées dans l'inode
 - Longueur du fichier
 - propriétaire
 - Mode (type de fichier, droits d'accès)
 - Timestamps

Etapes pour trouver le fichier /usr/ast/mbox

Root directory

1	.
1	..
4	bin
7	dev
14	lib
9	etc
6	usr
8	tmp

Looking up
usr yields
i-node 6

I-node 6
is for /usr

Mode size times
132

I-node 6
says that
/usr is in
block 132

Block 132
is /usr
directory

6	•
1	••
19	dick
30	erik
51	jim
26	ast
45	bal

/usr/ast
is i-node
26

I-node 26
is for
/usr/ast

Mode size times
406

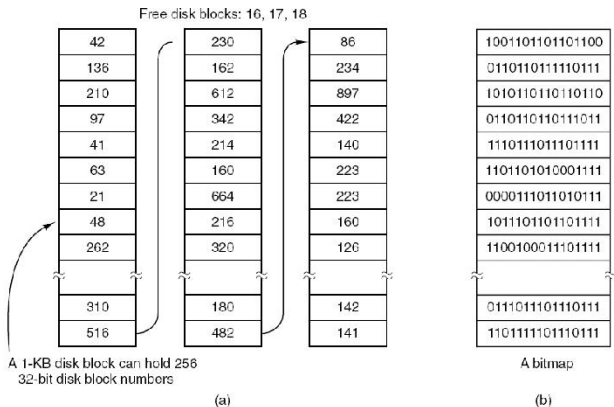
I-node 26
says that
/usr/ast is in
block 406

Block 406
is /usr/ast
directory

26	•
6	••
64	grants
92	books
60	mbox
81	minix
17	src

/usr/ast/mbox
is i-node
60

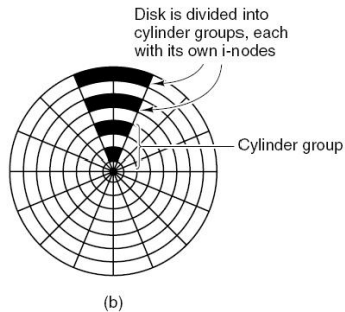
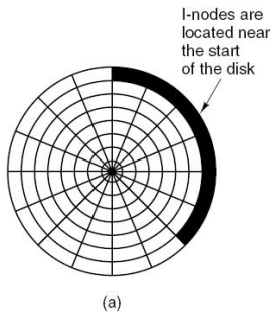
- On peut maintenir la liste des blocs libres grâce à une liste chaînée ou des bitmaps



- Bonne nouvelle: s'il y a beaucoup de blocs libres alors on peut en utiliser certains pour stocker la liste des blocs libres

Choix des blocs libres

- Quand on écrit dans les fichiers il faut **choisir des blocs libres** pour stocker les données
 - Peut-on choisir intelligemment?
- Vieux systèmes de fichiers:
 - Garde tous les inodes au début du disque
 - Le reste du disque contient les blocs de données
 - Mais n'importe quel accès disque demande d'**accéder à l'inode et aux blocs de données!**



Choisir une bonne taille de blocs

- On n'est pas obligés d'utiliser 1 bloc du système de fichiers = 1 secteur disque
 - On pourrait par exemple utiliser 1 bloc = 7 secteurs $\Rightarrow$ 1 bloc = 3.5 kB
- Grandes tailles de blocs:
 - Bonne performance d'accès (on utilise beaucoup de blocs contigus)
 - Gaspillage d'espace disque sur le dernier bloc
- Petites tailles de blocs:
 - Moins bonne performance
 - Mais meilleure utilisation de la capacité de stockage
- Si on suppose que tous les fichiers font exactement 2 kB:

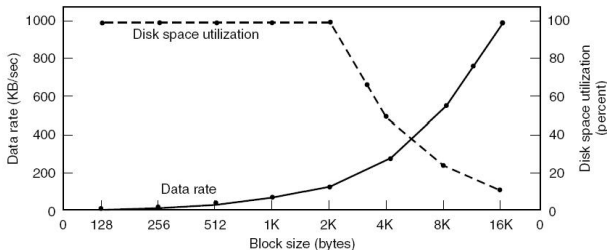


Table of Contents

- 1 Introduction
- 2 Implémentation des systèmes de fichiers
- 3 Technologies RAID**
- 4 Conclusion

- Les disques ont **trois MAUVAISES propriétés**:
 - 1 Ils sont **petits** (comme je stocke mes 100 TB de données?)
 - 2 Ils sont **lents** (forte latence, faible bande passante; **les performances augmentent moins vite que celle des CPU**)
 - 3 Ils sont **fragiles** (on doit remplacer 2-4% des disques chaque année, davantage dans des environnements exigeants)
- Les solutions simples ne marchent pas:
 - 1 Disques plus grands $\Rightarrow$ trop cher
 - 2 Disques plus rapides $\Rightarrow$ trop cher
 - 3 Disques plus solides $\Rightarrow$ trop cher
 - 4 Disques plus grands, plus rapides et plus solides $\Rightarrow$ BEAUCOUP trop cher!

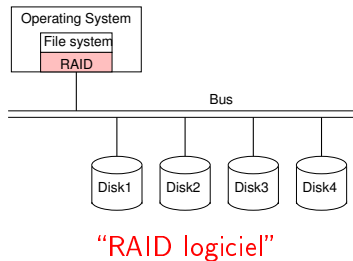
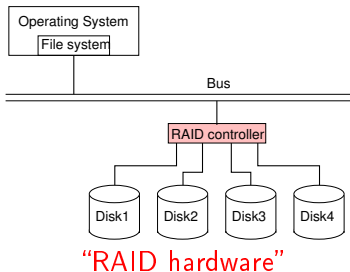
Est-ce qu'on peut résoudre les trois problèmes en même temps?
(et sans payer trop cher?)

- Les disques ont **trois MAUVAISES propriétés**:
 - 1 Ils sont **petits** (comme je stocke mes 100 TB de données?)
 - 2 Ils sont **lents** (forte latence, faible bande passante; **les performances augmentent moins vite que celle des CPU**)
 - 3 Ils sont **fragiles** (on doit remplacer 2-4% des disques chaque année, davantage dans des environnements exigeants)
- Les solution simples ne marchent pas:
 - 1 Disques plus grands $\Rightarrow$ trop cher
 - 2 Disques plus rapides $\Rightarrow$ trop cher
 - 3 Disques plus solides $\Rightarrow$ trop cher
 - 4 Disques plus grands, plus rapides et plus solides $\Rightarrow$ BEAUCOUP trop cher!

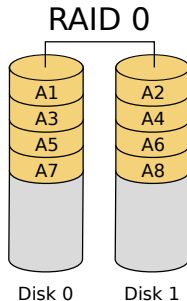
Est-ce qu'on peut résoudre les trois problèmes en même temps?
(et sans payer trop cher?)

Utilisons **plusieurs disques bon marché** au lieu **d'un seul disque très cher...**

- RAID = **Redundant Array of Inexpensive/Independent Disks**
 - On utilise plusieurs disques ensemble **c'est pas une partition**
 - Et on les traite comme une seule entité
- RAID peut être implémenté dans le contrôleur des disques (**RAID hardware**) ou dans le système d'exploitation (**RAID logiciel**)

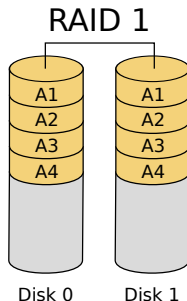


- **Striping** = distribuer les secteurs entre plusieurs disques
 - Si le programme accède à plusieurs secteurs consécutifs on peut faire les accès en parallèle $\Rightarrow$ meilleure performance
 - Pas de redondance $\Rightarrow$ pas de gain de fiabilité



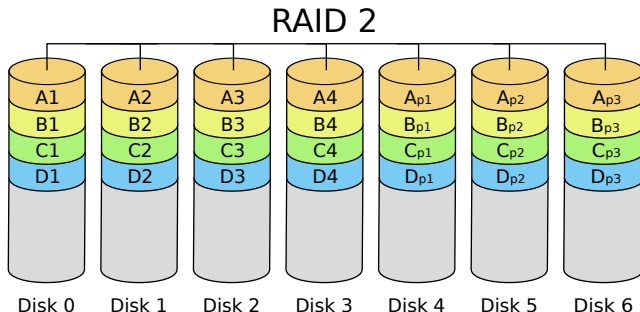
RAID 1: réplication

- **Réplication** = création de plusieurs copies identiques sur des disques différents
 - On peut faire les lectures depuis n'importe quelle copie $\Rightarrow$ bonnes performances en lecture
 - Il faut faire les écritures sur tous les disques $\Rightarrow$ pas de gain de performance en écriture
 - Si un disque se plante on peut accéder aux données sur les autres disques $\Rightarrow$ bonne fiabilité
 - On utilise **N fois plus d'espace disque!** $\Rightarrow$ assez cher



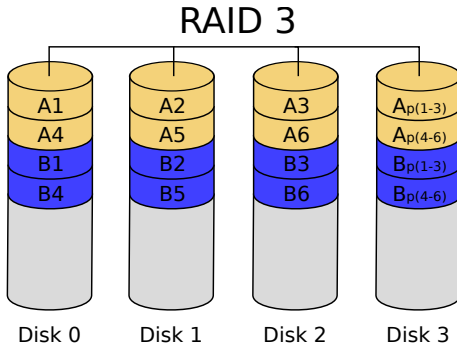
RAID 2: Code de Hamming

- Un code de Hamming permet de **détecter** et de **corriger** des erreurs, mais avec moins de redondance que la réplication
 - Les bits de chaque octets sont **distribués** sur plusieurs disques
 - Et on stocke le code de Hamming sur des disques à part
 - Cela permet de détecter et corriger des erreurs d'un bit



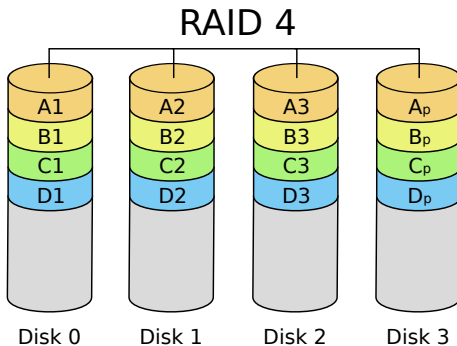
RAID 3: bits de parité

- Plus simple que le code de Hamming: on utilise des **bits de parité**
- Cela permet de **détecter les erreurs** (mais pas de les réparer)
- En cas de plantage du disque, on sait où est l'erreur, on peut la réparer $\Rightarrow$ bonne fiabilité



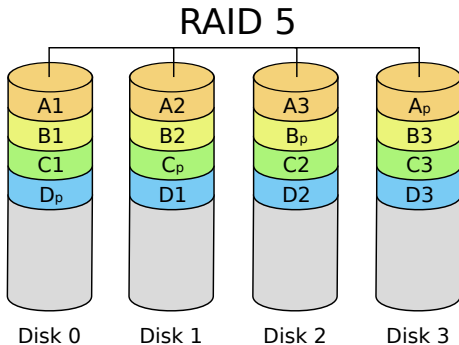
RAID 4: parité au niveau des blocs

- Similaire à RAID3, mais il fonctionne **bloc par bloc**
- Mêmes bonnes propriétés que RAID3
- Une nouvelle bonne propriété: on peut de nouveau accélérer les lectures/écritures
- Mais: pour chaque petite écriture il faut lire tous les blocs pour régénérer les blocs de parité



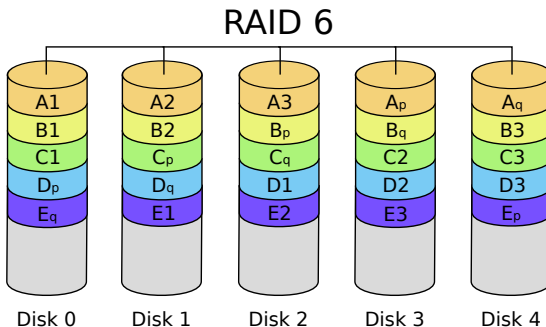
RAID 5: distribution symétrique des blocs de parité

- Similaire à RAID4, mais les blocs de données et de parité sont stockés sur les mêmes disques
- Cela répartit la charge de calculer les bits de parité (au moins avec du RAID hardware)



RAID 6

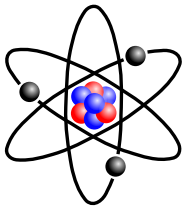
- Similaire à RAID 5 mais avec **deux fois plus de blocs de parité**
- On peut supporter la perte de **deux disques à la fois**



Les configurations RAID les plus utilisées sont RAID 1 (réplication totale), RAID 5 (avec de la redondance), RAID 6 (avec une double dose de redondance).

Table of Contents

- 1 Introduction
- 2 Implémentation des systèmes de fichiers
- 3 Technologies RAID
- 4 Conclusion**



Oui bien sûr...



```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, world!");  
    }  
}
```

Oui bien sûr...



Hello, world!